

Phase 4 Structured Note: Step-by-Step AVL Creation and AVL vs Standard BST

1. Where We Are Now

Before Phase 4, you already learned the main building blocks of AVL trees.

From Phase 1

You learned how to detect whether a tree is balanced.

An AVL tree is a Binary Search Tree, or BST, with one extra rule:

Every node must stay balanced.

A BST follows this rule:

left subtree values < node value < right subtree values

An AVL tree also checks the balance factor of every node:

Balance Factor = height(left subtree) - height(right subtree)

Valid AVL balance factors are:

-1, 0, +1

Invalid balance factors are:

+2, -2, +3, -3, ...

So Phase 1 answered this question:

Is the tree balanced or unbalanced?

From Phase 2

You learned that AVL trees fix imbalance using rotations.

The four insertion rotation cases are:

Case	Shape	Rotation type
LL	Left-left straight line	Single rotation
RR	Right-right straight line	Single rotation
LR	Left-right zig-zag	Double rotation
RL	Right-left zig-zag	Double rotation

So Phase 2 answered this question:

If the tree is unbalanced, which rotation fixes it?

From Phase 3

You learned how AVL insertion is written in C.

The insertion process is:

1. Insert like a normal BST.
2. Return upward through recursion.
3. Update height.
4. Check balance factor.
5. Apply LL, RR, LR, or RL rotation if needed.
6. Return the new root of the subtree.

So Phase 3 answered this question:

How do rotations work safely in C using pointers and height updates?

What Phase 4 Adds

Phase 4 combines everything.

Instead of studying one small rotation at a time, Phase 4 builds a full AVL tree from a real sequence of values:

10, 20, 30, 25, 28, 27, 5

You will see how several rotations can happen during one complete AVL creation process.

Phase 4 also compares the final AVL tree with a normal BST built from the same values.

The main idea is:

Phase 4 teaches you how to trace a complete AVL insertion sequence from start to finish.

2. Main Goal of Phase 4

The main goal of Phase 4 is to make AVL creation feel practical.

By the end of this phase, you should be able to:

- Insert values into an AVL tree one by one.
- Update heights after each insertion.
- Calculate balance factors after each insertion.
- Find the first imbalanced ancestor from the bottom.
- Choose the correct rotation case.
- Apply RR, LR, RL, and LL rotations during one full build.
- Understand why AVL trees stay shorter than normal BSTs.
- Compare search cost in an AVL tree and a normal BST.

Memory line:

AVL insertion = BST insertion + bottom-up balancing.

3. Key Principle: Fix Bottom-Up

After inserting a new value, not every node in the tree is affected.

Only the nodes on the path from the inserted node back to the root can have changed height.

That is why AVL insertion checks balance while moving upward.

Beginner rule

After insertion, start checking from the inserted node and move upward.

The first node you meet with balance factor `+2` or `-2` is the first imbalanced ancestor.

That node is the local root of the problem.

Why this matters

A beginner may want to check the root first, but this is not the best way to think about AVL insertion.

AVL insertion fixes the lowest broken part first.

That local fix usually repairs the height problem for the rest of the path.

Memory line:

Fix the first imbalanced ancestor from the bottom.

4. Important Rules Used in This Phase

4.1 BST insertion rule

When inserting a value:

smaller value goes left
greater value goes right

4.2 Height rule

In this curriculum:

NULL child height = 0
leaf node height = 1

Height formula:

```
height(node) = 1 + max(height(left), height(right))
```

4.3 Balance factor rule

Balance factor formula:

```
BF = height(left subtree) - height(right subtree)
```

Balance meaning:

Balance factor	Meaning	AVL status
+1	Left side is taller by 1	Valid
0	Both sides have same height	Valid
-1	Right side is taller by 1	Valid
+2	Left side is too tall	Invalid
-2	Right side is too tall	Invalid

4.4 Rotation decision rule

Problem	Path from imbalanced node	Case	Fix
Left-heavy	left → left	LL	Single rotation
Right-heavy	right → right	RR	Single rotation
Left-heavy zig-zag	left → right	LR	Double rotation
Right-heavy zig-zag	right → left	RL	Double rotation

5. Insertion Sequence for Phase 4

The full sequence is:

```
10, 20, 30, 25, 28, 27, 5
```

We will insert these one at a time.

After each insertion, we will ask:

1. Where does the value go using the BST rule?
2. Which heights changed?
3. Which balance factors changed?
4. Is there an imbalanced node?
5. If yes, which rotation is needed?

6. Step-by-Step AVL Creation

Step 1: Insert 10

The tree is empty, so 10 becomes the root.

10

Node 10 has no children.

```
left height = 0
right height = 0
height(10) = 1
BF(10) = 0 - 0 = 0
```

BF(10) = 0, so the tree is balanced.

Rotation needed?

No rotation.

Step 2: Insert 20

Use the BST rule.

20 is greater than 10, so it goes to the right of 10.

10

20

Now update bottom-up.

Node 20

20 is a leaf.

```
height(20) = 1
BF(20) = 0
```

Node 10

Node 10 has no left child and right child 20.

```
left height = 0
right height = 1
height(10) = 1 + max(0, 1) = 2
BF(10) = 0 - 1 = -1
```

BF(10) = -1, which is valid.

Rotation needed?

No rotation.

Step 3: Insert 30

Use the BST rule.

- 30 > 10, go right.
- 30 > 20, go right.

Before fixing:

10

20

30

Now update bottom-up.

Node 30

30 is a leaf.

```
height(30) = 1
BF(30) = 0
```

Node 20

Node 20 has right child 30.

```
left height = 0
right height = 1
height(20) = 2
BF(20) = 0 - 1 = -1
```

BF(20) = -1, so node 20 is balanced.

Node 10

Node 10 has right subtree height 2.

```
left height = 0
right height = 2
height(10) = 3
BF(10) = 0 - 2 = -2
```

BF(10) = -2, so node 10 is imbalanced.

Identify the case

From the imbalanced node 10 to the inserted node 30:

```
10 -> right -> 20 -> right -> 30
```

Direction:

right -> right

This is an **RR case**.

Fix

Use an RR rotation at node **10**.

Before RR rotation:

```
10
  /
 20
  /
 30
```

After RR rotation:

```
  20
 /
10  30
```

Now all nodes are balanced.

Rotation used

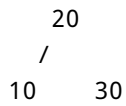
RR rotation

New local root

20

Step 4: Insert 25

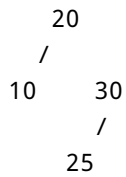
Current tree:



Use the BST rule.

- $25 > 20$, go right.
- $25 < 30$, go left.

After insertion:



Now update bottom-up.

Node 25

25 is a leaf.

```

height(25) = 1
BF(25) = 0
  
```

Node 30

Node 30 has left child 25.

```

left height = 1
right height = 0
height(30) = 2
BF(30) = 1 - 0 = +1
  
```

$BF(30) = +1$, so node 30 is balanced.

Node 20

Node 20 has left child 10 and right subtree rooted at 30.

```
left height = 1
right height = 2
height(20) = 3
BF(20) = 1 - 2 = -1
```

BF(20) = -1, so node 20 is balanced.

Rotation needed?

No rotation.

Step 5: Insert 28

Current tree:

```
  20
 /
10  30
   /
  25
```

Use the BST rule.

- 28 > 20, go right.
- 28 < 30, go left.
- 28 > 25, go right.

Before fixing:

```
  20
 /
10  30
   /
  25
   \
   28
```

Now update bottom-up.

Node 28

28 is a leaf.

```
height(28) = 1
BF(28) = 0
```

Node 25

Node 25 has right child 28.

```
left height = 0
right height = 1
height(25) = 2
BF(25) = 0 - 1 = -1
```

BF(25) = -1, so node 25 is balanced.

Node 30

Node 30 has left subtree height 2 and no right child.

```
left height = 2
right height = 0
height(30) = 3
BF(30) = 2 - 0 = +2
```

BF(30) = +2, so node 30 is imbalanced.

This is the first imbalanced ancestor from the bottom.

Identify the case

From the imbalanced node 30 to the inserted node 28:

```
30 -> left -> 25 -> right -> 28
```

Direction:

```
left -> right
```

This is an **LR case**.

Fix

Use an LR rotation at node **30**.

Before local LR rotation:

```
  30
 /
25
  28
```

After local LR rotation:

```
  28
 /
25  30
```

Now attach this repaired subtree back to the main tree.

The whole tree becomes:

```
  20
 /
10  28
   /
  25  30
```

Rotation used

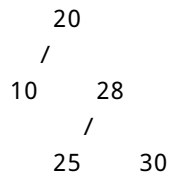
LR rotation

New local root

28

Step 6: Insert 27

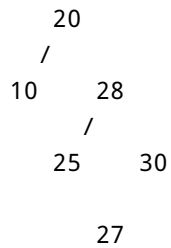
Current tree:



Use the BST rule.

- $27 > 20$, go right.
- $27 < 28$, go left.
- $27 > 25$, go right.

Before fixing:



Now update bottom-up.

Node 27

27 is a leaf.

```
height(27) = 1
BF(27) = 0
```

Node 25

Node 25 has right child 27.

```
left height = 0
right height = 1
```

```
height(25) = 2
BF(25) = 0 - 1 = -1
```

BF(25) = -1, so node 25 is balanced.

Node 28

Node 28 has left subtree rooted at 25 and right child 30.

```
left height = 2
right height = 1
height(28) = 3
BF(28) = 2 - 1 = +1
```

BF(28) = +1, so node 28 is balanced.

Node 20

Node 20 has left child 10 and right subtree rooted at 28.

```
left height = 1
right height = 3
height(20) = 4
BF(20) = 1 - 3 = -2
```

BF(20) = -2, so node 20 is imbalanced.

This is the first imbalanced ancestor from the bottom.

Identify the case

From the imbalanced node 20 to the inserted node 27:

```
20 -> right -> 28 -> left -> 25 -> right -> 27
```

The first two directions from the imbalanced node are:

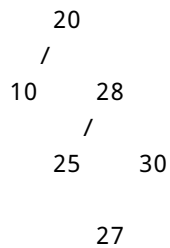
```
right -> left
```

This is an **RL case**.

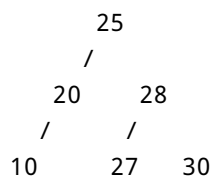
Fix

Use an RL rotation at node **20**.

Before local RL rotation:



After RL rotation:



The node **25** becomes the new root of this local subtree.

Because this local subtree was the whole tree, **25** also becomes the new global root.

Rotation used

RL rotation

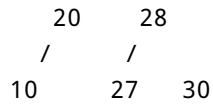
New global root

25

Step 7: Insert 5

Current tree:

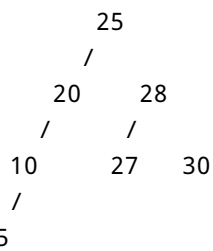




Use the BST rule.

- $5 < 25$, go left.
- $5 < 20$, go left.
- $5 < 10$, go left.

Before fixing:



Now update bottom-up.

Node 5

5 is a leaf.

```

height(5) = 1
BF(5) = 0
  
```

Node 10

Node 10 has left child 5.

```

left height = 1
right height = 0
height(10) = 2
BF(10) = 1 - 0 = +1
  
```

$BF(10) = +1$, so node 10 is balanced.

Node 20

Node 20 has left subtree rooted at 10 and no right child.

```
left height = 2
right height = 0
height(20) = 3
BF(20) = 2 - 0 = +2
```

BF(20) = +2, so node 20 is imbalanced.

This is the first imbalanced ancestor from the bottom.

Identify the case

From the imbalanced node 20 to the inserted node 5:

```
20 -> left -> 10 -> left -> 5
```

Direction:

```
left -> left
```

This is an **LL case**.

Fix

Use an LL rotation at node 20.

Before local LL rotation:

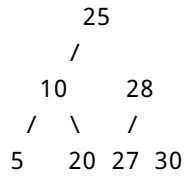
```
  20
 /
10
 /
5
```

After local LL rotation:

```
  10
 /
5   20
```

Attach this repaired subtree back to the main tree.

Final AVL tree:



Rotation used

LL rotation

New local root

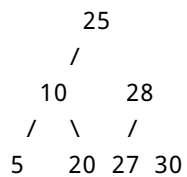
10

7. Final AVL Tree

After inserting all values:

10, 20, 30, 25, 28, 27, 5

The final AVL tree is:



This tree is balanced and short.

8. Final Balance Factor Check

Let us check every node.

Leaf nodes

Nodes 5, 20, 27, and 30 are leaves.

Each has:

```
left height = 0
right height = 0
height = 1
BF = 0
```

So:

```
BF(5) = 0
BF(20) = 0
BF(27) = 0
BF(30) = 0
```

Node 10

Node 10 has left child 5 and right child 20.

```
left height = 1
right height = 1
height(10) = 2
BF(10) = 1 - 1 = 0
```

Node 28

Node 28 has left child 27 and right child 30.

```
left height = 1
right height = 1
height(28) = 2
BF(28) = 1 - 1 = 0
```

Node 25

Node 25 has left subtree rooted at 10 and right subtree rooted at 28.

```
left height = 2
right height = 2
height(25) = 3
BF(25) = 2 - 2 = 0
```

Final balance factor list

```
BF(5)  = 0
BF(20) = 0
BF(10) = 0
BF(27) = 0
BF(30) = 0
BF(28) = 0
BF(25) = 0
```

Every balance factor is valid.

So the final tree is AVL balanced.

9. Rotation Events in This Build

During this insertion sequence, we saw several rotation cases.

Inserted value	First imbalanced node	Direction	Case	Rotation
30	10	right → right	RR	RR rotation
28	30	left → right	LR	LR rotation
27	20	right → left	RL	RL rotation
5	20	left → left	LL	LL rotation

This is why Phase 4 is useful: it shows all major insertion rotation ideas inside one realistic example.

10. Why We Fix the First Imbalanced Ancestor

After an insertion, the new node can only affect its ancestors.

For example, when inserting `28`, the path is:

```
20 -> 30 -> 25 -> 28
```

Only these nodes can have changed height:

```
25, 30, 20
```

When checking upward, node `30` is the first node that becomes imbalanced.

So we rotate at `30`, not at `20`.

Memory line:

```
The first broken ancestor from the bottom is the local problem.
```

11. AVL Tree vs Normal BST

Now compare the AVL tree with a normal BST built from the same sequence.

The same values are inserted:

```
10, 20, 30, 25, 28, 27, 5
```

A normal BST follows only this rule:

```
smaller goes left  
greater goes right
```

A normal BST does not rotate.

11.1 Normal BST Built From the Same Values

Insert `10`:

```
10
```

Insert `20`:

10

20

Insert 30 :

10

20

30

Insert 25 :

10

20

30

/

25

Insert 28 :

10

20

30

/

25

28

Insert 27 :

10

20

30

/

```

25
  /
 28
 /
27

```

Insert 5:

```

10
 /
5  20
    /
    30
   /
   25
    /
    28
   /
   27

```

This is still a valid BST.

But it is taller and less balanced than the AVL tree.

11.2 Final AVL Tree Again

The AVL tree from the same values is:

```

25
 /
10 28
 / \ /
5 20 27 30

```

This AVL tree is shorter and more even.

12. Why the AVL Tree Is Better

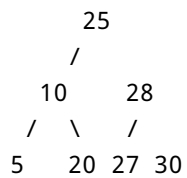
A shorter tree usually means fewer comparisons during search.

The AVL tree keeps the height small by rotating when needed.

A normal BST can become tall if the insertion order is unlucky.

12.1 Searching for 27 in the AVL Tree

Final AVL tree:



Search path for 27 :

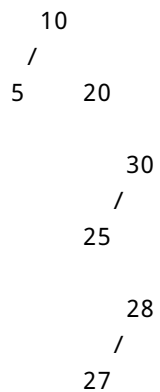
25 -> 28 -> 27

Number of comparisons:

3 comparisons

12.2 Searching for 27 in the Normal BST

Normal BST:



Search path for 27 :

10 -> 20 -> 30 -> 25 -> 28 -> 27

Number of comparisons:

6 comparisons

The normal BST needs more comparisons because it is taller.

12.3 Main Comparison

Tree type	Shape	Search performance
Normal BST	Can become tall	Can degrade to $O(N)$
AVL tree	Stays balanced	Stays $O(\log N)$

Memory line:

AVL trees rotate to stay short.

13. Important Concept: Rotations Do Not Change BST Order

Rotations change the shape of the tree, but they do not change the sorted order of the values.

For example, before and after rotation, the inorder traversal remains sorted.

Final AVL tree:

```
      25
     /
    10  28
   /  \ /
  5   20 27 30
```

Inorder traversal:

5, 10, 20, 25, 27, 28, 30

That is sorted.

So the tree is still a valid BST.

Memory line:

Rotations change shape, not sorted order.

14. Complete C Program for Phase 4

This program builds both:

1. An AVL tree.
2. A normal BST.

It uses the same insertion sequence for both trees:

10, 20, 30, 25, 28, 27, 5

Then it prints:

- AVL preorder.
- BST preorder.
- AVL height.
- BST height.
- Comparisons to find 27 in each tree.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    int height;
    struct Node *left;
    struct Node *right;
} Node;

int max(int a, int b) {
    return (a > b) ? a : b;
```

```

}

int h(Node *p) {
    return p ? p->height : 0;
}

int bf(Node *p) {
    return p ? h(p->left) - h(p->right) : 0;
}

Node *newNode(int key) {
    Node *p = (Node *)malloc(sizeof(Node));
    p->data = key;
    p->height = 1;
    p->left = NULL;
    p->right = NULL;
    return p;
}

void update(Node *p) {
    if (p) {
        p->height = 1 + max(h(p->left), h(p->right));
    }
}

Node *ll(Node *p) {
    Node *pl = p->left;
    Node *plr = pl->right;

    pl->right = p;
    p->left = plr;

    update(p);
    update(pl);

    return pl;
}

Node *rr(Node *p) {
    Node *pr = p->right;
    Node *prl = pr->left;

    pr->left = p;
    p->right = prl;

    update(p);
    update(pr);
}

```

```

        return pr;
    }

Node *lr(Node *p) {
    Node *pl = p->left;
    Node *plr = pl->right;

    pl->right = plr->left;
    p->left = plr->right;
    plr->left = pl;
    plr->right = p;

    update(pl);
    update(p);
    update(plr);

    return plr;
}

Node *rl(Node *p) {
    Node *pr = p->right;
    Node *prl = pr->left;

    pr->left = prl->right;
    p->right = prl->left;
    prl->right = pr;
    prl->left = p;

    update(pr);
    update(p);
    update(prl);

    return prl;
}

Node *insertAVL(Node *p, int key) {
    if (p == NULL) {
        return newNode(key);
    }

    if (key < p->data) {
        p->left = insertAVL(p->left, key);
    } else if (key > p->data) {
        p->right = insertAVL(p->right, key);
    } else {
        return p;
    }
}

```

```

    update(p);

    if (bf(p) == 2 && bf(p->left) >= 0) {
        return ll(p);
    }

    if (bf(p) == 2 && bf(p->left) < 0) {
        return lr(p);
    }

    if (bf(p) == -2 && bf(p->right) <= 0) {
        return rr(p);
    }

    if (bf(p) == -2 && bf(p->right) > 0) {
        return rl(p);
    }

    return p;
}

Node *insertBST(Node *p, int key) {
    if (p == NULL) {
        return newNode(key);
    }

    if (key < p->data) {
        p->left = insertBST(p->left, key);
    } else if (key > p->data) {
        p->right = insertBST(p->right, key);
    }

    p->height = 1 + max(h(p->left), h(p->right));
    return p;
}

int comparisonsToFind(Node *p, int key) {
    int count = 0;

    while (p != NULL) {
        count++;

        if (key == p->data) {
            return count;
        }

        if (key < p->data) {
            p = p->left;
        }
    }
}

```

```

        } else {
            p = p->right;
        }
    }

    return count;
}

void preorder(Node *p) {
    if (p == NULL) {
        return;
    }

    printf("%d(h=%d,bf=%d) ", p->data, p->height, bf(p));
    preorder(p->left);
    preorder(p->right);
}

int main(void) {
    int keys[] = {10, 20, 30, 25, 28, 27, 5};
    int n = sizeof(keys) / sizeof(keys[0]);

    Node *avl = NULL;
    Node *bst = NULL;

    for (int i = 0; i < n; i++) {
        avl = insertAVL(avl, keys[i]);
        bst = insertBST(bst, keys[i]);
    }

    printf("AVL preorder: ");
    preorder(avl);
    printf("\n");

    printf("BST preorder: ");
    preorder(bst);
    printf("\n");

    printf("AVL height = %d\n", h(avl));
    printf("BST height = %d\n", h(bst));

    printf("Comparisons to find 27 in AVL = %d\n", comparisonsToFind(avl, 27));
    printf("Comparisons to find 27 in BST = %d\n", comparisonsToFind(bst, 27));

    return 0;
}

```

15. Explanation of the C Program

15.1 Node structure

```
typedef struct Node {  
    int data;  
    int height;  
    struct Node *left;  
    struct Node *right;  
} Node;
```

Each node stores:

Field	Meaning
data	The value in the node
height	Height of the subtree rooted at this node
left	Pointer to left child
right	Pointer to right child

15.2 Height function

```
int h(Node *p) {  
    return p ? p->height : 0;  
}
```

This means:

- If the node exists, return its height.
- If the node is `NULL`, return `0`.

This matches the rule:

```
NULL child height = 0
```


15.3 Balance factor function

```
int bf(Node *p) {  
    return p ? h(p->left) - h(p->right) : 0;  
}
```

This follows the formula:

```
BF = height(left subtree) - height(right subtree)
```

15.4 AVL insertion

The AVL insertion function first inserts like a normal BST:

```
if (key < p->data) {  
    p->left = insertAVL(p->left, key);  
} else if (key > p->data) {  
    p->right = insertAVL(p->right, key);  
}
```

Then it updates height:

```
update(p);
```

Then it checks which rotation is needed:

```
if (bf(p) == 2 && bf(p->left) >= 0) return ll(p);  
if (bf(p) == 2 && bf(p->left) < 0) return lr(p);  
if (bf(p) == -2 && bf(p->right) <= 0) return rr(p);  
if (bf(p) == -2 && bf(p->right) > 0) return rl(p);
```

Rotation decision table from the code

Condition	Meaning	Rotation
<code>bf(p) == 2 && bf(p->left) >= 0</code>	Left-left problem	LL
<code>bf(p) == 2 && bf(p->left) < 0</code>	Left-right problem	LR
<code>bf(p) == -2 && bf(p->right) <= 0</code>	Right-right problem	RR

Condition	Meaning	Rotation
<code>bf(p) == -2 && bf(p->right) > 0</code>	Right-left problem	RL

15.5 BST insertion

The normal BST insertion function does not rotate.

It only inserts using the BST rule:

```
if (key < p->data) {
    p->left = insertBST(p->left, key);
} else if (key > p->data) {
    p->right = insertBST(p->right, key);
}
```

Then it updates height only for comparison purposes.

It does not check balance factor.

It does not rotate.

15.6 Comparison counting

This function counts how many nodes we visit while searching:

```
int comparisonsToFind(Node *p, int key)
```

Every visited node counts as one comparison.

This helps us see why AVL trees can be faster.

16. Program Dry Run in Simple Words

The program inserts the same keys into both trees:

```
10, 20, 30, 25, 28, 27, 5
```

For the AVL tree:

- It inserts like a BST.
- It updates height.
- It checks balance.
- It rotates when needed.

For the normal BST:

- It inserts like a BST.
- It does not rotate.

At the end, the AVL tree is shorter.

The normal BST is taller.

So searching in the AVL tree needs fewer comparisons.

17. Time Complexity

17.1 AVL insertion

AVL insertion takes:

$O(\log N)$

Why?

Because AVL trees stay balanced.

The height remains logarithmic.

Insertion follows one path down and one path back up.

The path length is $O(\log N)$.

Rotations are only $O(1)$, because they change a fixed number of pointers.

So total AVL insertion is:

$O(\log N)$

17.2 Normal BST insertion

Normal BST insertion takes:

$O(H)$

where H is the height of the tree.

If the BST is balanced, H may be $\log N$.

But if the BST becomes tall like a chain, H may become N .

So worst-case BST insertion is:

$O(N)$

17.3 Searching

Tree	Search time
AVL tree	$O(\log N)$
Normal BST	$O(H)$, worst case $O(N)$

17.4 Building the whole tree

If you insert N values into an AVL tree:

$O(N \log N)$

If you insert N values into a normal BST:

$O(NH)$

In the worst case, if the normal BST becomes like a chain:

$O(N^2)$

18. Common Beginner Mistakes in Phase 4

Mistake 1: Fixing the root first

Wrong idea:

Always check the root first after insertion.

Correct idea:

Check from the inserted node upward.

Fix the first imbalanced ancestor from the bottom.

Mistake 2: Forgetting that AVL insertion starts like BST insertion

Wrong idea:

AVL insertion is completely different from BST insertion.

Correct idea:

AVL insertion first follows the normal BST rule.

Then AVL adds height updates, balance checks, and rotations.

Mistake 3: Thinking rotations change the sorted order

Wrong idea:

After rotation, the tree is no longer a BST.

Correct idea:

Rotations preserve BST order.

The inorder traversal remains sorted.

Mistake 4: Choosing the wrong rotation case

Example:

If the parent is right-heavy, a beginner may immediately choose RR.

But this is not always correct.

You must check the path.

- right → right means RR
- right → left means RL

Same idea for left-heavy cases:

- left → left means LL
 - left → right means LR
-

Mistake 5: Comparing height numbers without knowing the convention

Some teachers count height by edges.

This curriculum counts height by node levels:

```
NULL height = 0  
leaf height = 1
```

So always know the convention being used.

Mistake 6: Thinking a normal BST and AVL tree store different values

Wrong idea:

AVL changes the values.

Correct idea:

AVL stores the same values, but in a better shape.

The values are the same.

The structure is different.

19. Quick Reference: Phase 4 Insertion Table

Step	Inserted value	Tree problem	Rotation	Result
1	10	One node only	None	10 is root
2	20	BF(10) = -1	None	Still balanced
3	30	RR at 10	RR	20 becomes root
4	25	All BF values valid	None	Still balanced
5	28	LR at 30	LR	28 becomes local root
6	27	RL at 20	RL	25 becomes global root
7	5	LL at 20	LL	10 becomes local root

20. Quick Reference: Final Trees

Final AVL tree

```
      25
     /
    10  28
```

```
  /  \  /
5    20 27 30
```

Final normal BST

```
    10
   /
  5   20
      /
     30
    /
   25
    /
   28
  /
 27
```

21. Phase 4 Mental Model

Think of AVL insertion like this:

```
Insert the value like a normal BST.
Then walk back upward.
At each node, ask:
1. What is my new height?
2. What is my balance factor?
3. Am I still balanced?
4. If not, which direction caused the problem?
5. Which rotation fixes the problem?
```

If the node is balanced, keep moving upward.

If the node is imbalanced, rotate there.

Memory line:

```
AVL trees repair the first broken place on the way back up.
```


22. Practice Questions

Question 1

In the sequence `10, 20, 30`, which node becomes imbalanced?

Answer:

10

Question 2

In the sequence `10, 20, 30`, what rotation is needed?

Answer:

RR rotation

Question 3

When inserting `28`, which node becomes the first imbalanced ancestor?

Answer:

30

Question 4

When inserting `28`, what is the rotation case?

Answer:

LR case

Because the direction from `30` to `28` is:

left -> right

Question 5

When inserting 27, which node becomes imbalanced?

Answer:

20

Question 6

When inserting 27, what is the rotation case?

Answer:

RL case

Because the direction from 20 to 27 starts:

right -> left

Question 7

When inserting 5, what is the rotation case?

Answer:

LL case

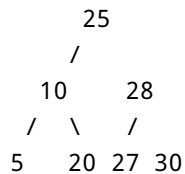
Because the direction from 20 to 5 is:

left -> left

Question 8

What is the final AVL tree?

Answer:



Question 9

Why is the AVL tree better than the normal BST?

Answer:

The AVL tree stays shorter, so search needs fewer comparisons.

Question 10

Does AVL rotation change the sorted order of values?

Answer:

No. Rotation changes shape, not sorted order.

23. One-Sentence Memory Lines

Phase 4 shows how to build a full AVL tree step by step.

AVL insertion starts as normal BST insertion.

After insertion, update height and check balance from bottom to top.

Fix the first imbalanced ancestor from the bottom.

RR happens when the path is right -> right.

LR happens when the path is left -> right.

RL happens when the path is right -> left.

LL happens when the path is left -> left.

AVL rotations keep the tree short.

AVL trees and normal BSTs store the same values, but AVL trees keep a better shape.

Rotations change shape, not sorted order.

A shorter tree means fewer comparisons during search.

24. Final Phase 4 Summary

Phase 4 teaches complete AVL tree creation.

You inserted this sequence:

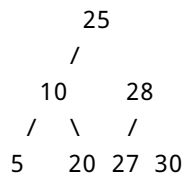
10, 20, 30, 25, 28, 27, 5

During the build, the AVL tree used rotations to stay balanced.

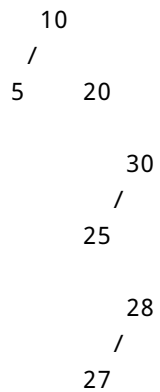
The rotation events were:

Insert 30 -> RR rotation
Insert 28 -> LR rotation
Insert 27 -> RL rotation
Insert 5 -> LL rotation

The final AVL tree was:



A normal BST built from the same values became taller:



Searching for 27 in the AVL tree takes fewer comparisons:

AVL: 25 -> 28 -> 27

Searching for 27 in the normal BST takes more comparisons:

BST: 10 -> 20 -> 30 -> 25 -> 28 -> 27

The big lesson is:

AVL trees stay efficient because they rebalance after insertion.

25. Phase 4 Mastery Checklist

You are ready for Phase 5 if you can:

- Explain why AVL insertion starts like BST insertion.
- Explain why height is updated bottom-up.
- Explain what the first imbalanced ancestor means.
- Insert 10, 20, 30, 25, 28, 27, 5 step by step.
- Identify the RR case after inserting 30.
- Identify the LR case after inserting 28.
- Identify the RL case after inserting 27.
- Identify the LL case after inserting 5.
- Draw the final AVL tree.
- Draw the normal BST from the same insertion sequence.
- Explain why the normal BST is taller.
- Explain why AVL search is faster in this example.
- Explain why rotations preserve BST order.
- Understand that AVL operations stay efficient because the tree height stays small.

If you can do these, you understand Phase 4 well.

26. Final Memory Line

Phase 4 is about watching AVL insertion in action: insert like a BST, check upward, rotate at the first broken ancestor, and keep the tree short.